Tortuquiz

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Melissa Bohomol	-	26029799
Pedro Gabriel de Miranda Soares	-	26029599
Caio Andrade Solera	-	26029890
Yohann Correia Cury	-	25028106

São Paulo

2026

Sumário

1 INTRODUÇÃO 4

1.1 – Processo de desenvolvimento de Software 4

1.2 Apresentação 4

2. DESIGN SPRINT – Ideação e prototipação do desafio 5

2.1 Desafio 5

2.2 Entender Mapear 5

2.2.1 – Entendimento e observação. 5

2.2.2 – COMO PODEMOS? 5

2.2.3 – Pesquisa Desk 5

2.3 Ideação – desenho da solução (trilha do usuário) 6

2.3.1 ”E se...?” → Geração de ideias doidas para inspirar ideias inovadoras 6

2.3.2 *Crazy 8s* (8 ideias em 8 minutos no papel) 6

2.3.3 Trilha do usuário 7

2.4 Prototipagem 8

3.REQUISITOS DE SISTEMA 8

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE 8

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE 9

4. DIAGRAMAS DE UML 16

4.1 CASOS DE USO 10

4.2 DIAGRAMA DE CLASSE 10

5. Especificação estrutura de uma rotina do sistema 10

5.1 – Apresentar a especificação de acordo com o modelo abaixo 10

5.2 – Apresentar a codificação da rotina 11

6. ARQUITETURA DO SISTEMA 11

6.1. Visão Geral do Sistema 11

6.2. Arquitetura Geral 11

6.3. Componentes Arquiteturais e Camadas 11

6.4. Diagrama da Arquitetura 12

6.5. Organização de Pastas e Módulos 13

6.6. Integrações Externas (se houver) 13

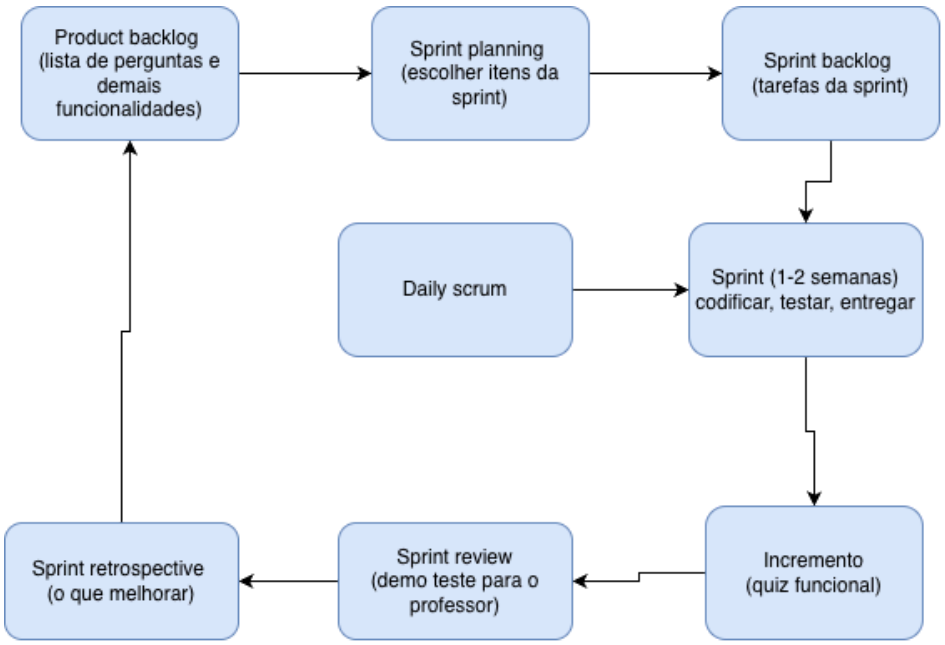
6.7. Ambiente e Implantação 14

8. REFERÊNCIAS BIBLIOGRÁFICAS 14

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo Scrum



1.2 Apresentação

O Tortuquiz é um jogo de perguntas e respostas de entretenimento, com a temática "Tortuguita", desenvolvido como aplicativo Desktop. O objetivo do jogador é avançar entre as perguntas, organizadas em três níveis de dificuldade (fácil, médio e difícil), acumulando pontos ao longo da partida. O projeto une diversão e aprendizado, pois também apresenta informações sobre as marcas e os produtos abordados nas perguntas. Fluxo do processo (simulação da partida): 1) Tela inicial e instruções: o aplicativo apresenta a identidade do projeto, o objetivo do jogo, as instruções de uso e a opção de iniciar uma partida. 2) Identificação do jogador: o jogador informa um nickname/apelido ou inicia de forma anônima, sem fornecer dados pessoais desnecessários. 3) Carga de conteúdo: o aplicativo recupera do Banco de Dados relacional as perguntas, alternativas, pistas, marcas/produtos e parâmetros de jogo. 4) Seleção do modo de jogo: o jogador escolhe a mecânica principal da partida, Quiz ou Descoberta por perguntas sucessivas, sendo a combinação das duas uma possível evolução do projeto. 5) Execução do quiz: o jogador responde às perguntas, progredindo do nível fácil ao difícil. 6) Pontuação e progresso: o aplicativo calcula a pontuação conforme a regra documentada e exibe o progresso da partida (rodada, quantidade de questões ou etapa atual). 7) Feedback educativo: após cada resposta, o aplicativo informa se o jogador acertou

ou errou e apresenta uma explicação sobre o tema. 8) Resultado final: ao término, o aplicativo exibe um resumo com a pontuação, o desempenho e as informações sobre as marcas/produtos abordados. 9) Histórico de partidas: o aplicativo registra as partidas realizadas, permitindo consultar os resultados anteriores.

DESIGN SPRINT – Ideação e prototipação do desafio

2.1. Desafio

Desenvolver o Tortuquiz, um aplicativo Desktop em C# de perguntas e respostas com a temática "Tortuguita". O jogador avança por perguntas de nível fácil, médio e difícil e acumula pontos. As perguntas, alternativas, pistas e informações sobre marcas/produtos ficam em um Banco de Dados relacional. O aplicativo também controla a identificação do jogador (nickname ou anônimo), a pontuação, o progresso, o feedback educativo, o resultado final e o histórico de partidas.

2.2. Entender Mapear

2.2.1. – Entendimento e observação.

Tempestade de ideias: perguntas por nível de dificuldade; pontuação maior em perguntas mais difíceis; pistas que o jogador pode usar; ranking de jogadores; ajuda como 50/50 ou pular pergunta; mascote tartaruga que reage aos acertos e erros; tempo limite por pergunta um minuto e meio; serão de 3 a 4 perguntas por nível; se o jogador erra uma resposta isso contabiliza no placar; o jogador deve acertar o máximo de perguntas o possível para ter uma melhor classificação; feedback com curiosidades após cada resposta; sons e animações.

Objetivo do projeto: criar um quiz de entretenimento em C# que ofereça uma experiência divertida. O jogador avança por níveis de dificuldade e aprende sobre marcas e produtos durante a partida.

2.2.2. – COMO PODEMOS?

O que já sabemos sobre os usuários? São jogadores casuais que gostam de jogos rápidos e querem aprender algo de forma leve. Esperamos que prefiram regras simples e feedback imediato.

Quais pesquisas, dados ou métricas estão disponíveis? Ainda não temos dados próprios. Podemos usar referências de quizzes conhecidos (Kahoot, Quizizz) e de pesquisas sobre gamificação no aprendizado, além de coletar feedback nos testes do protótipo.

Quais hipóteses iniciais temos? (1) Níveis progressivos mantêm o jogador engajado. (2) Feedback educativo após cada resposta aumenta o aprendizado. (3) Identificação simples, por apelido ou anônima, reduz a barreira para começar a jogar. (4) Um tema divertido (Tortuguita) aumenta o interesse.

Quem está envolvido no problema? Jogador, equipe de desenvolvimento, professor/orientador, administrador do conteúdo (quem cadastra as perguntas), as marcas/produtos abordadas e o sistema (aplicativo e banco de dados).

Mapa rápido de atores: Jogador → joga, escolhe o modo e vê o resultado. Administrador → cadastra e atualiza perguntas. Sistema → sorteia perguntas, calcula pontos e salva o histórico. Marcas/produtos → fornecem o conteúdo informativo exibido no feedback.

Como poderíamos tornar o aumento de dificuldade empolgante, e não frustrante, para o jogador?

Como poderíamos fazer o jogador aprender sobre as marcas/produtos sem que pareça uma propaganda?

Como poderíamos permitir que qualquer pessoa comece a jogar em poucos segundos, sem cadastro?

Como poderíamos usar o tema Tortuguita para tornar a experiência mais divertida e marcante?

Como poderíamos mostrar o progresso e o histórico de forma que motive o jogador a jogar de novo?

Como poderíamos facilitar o cadastro de novas perguntas no banco de dados sem mexer no código?

2.2.3. – Pesquisa Desk

Objetivo da pesquisa: reunir informações existentes para entender como quizzes digitais engajam jogadores, quais soluções já existem e quais boas práticas podemos aplicar no Tortuquiz.

O que a pesquisa busca

Tendências de gamificação e de jogos de perguntas e respostas (níveis de dificuldade, pontuação, ranking, ajudas e pistas).
Formas de dar feedback educativo que ajudem o jogador a aprender, e não só a acertar.
Boas práticas de usabilidade em jogos Desktop (telas simples, instruções claras, início rápido).

Formas de apresentar marcas e produtos sem parecer propaganda.

Concorrentes e benchmarks

Kahoot! e Quizizz: quizzes com pontuação por acerto e velocidade, ranking e feedback imediato. Como boa prática, usam cores, sons e progresso visível.
Perguntados / Trivia Crack: perguntas por categorias, personagens e mascotes, e progressão por fases. É a referência para o tema divertido e o uso de um mascote, como a Tortuguita.
Quiz de TV (estilo "Show do Milhão"): perguntas em níveis crescentes de dificuldade, com ajudas como pular ou eliminar alternativas. É a referência para o avanço do fácil ao difícil.
Diferencial do Tortuquiz: tema próprio, jogo Desktop leve, início anônimo sem cadastro, informações sobre marcas/produtos no feedback e histórico de partidas.

Boas práticas identificadas

Começar com perguntas fáceis e aumentar a dificuldade aos poucos, para manter o jogador motivado.
Mostrar o progresso (pergunta atual, pontuação) durante toda a partida.
Dar retorno imediato após cada resposta, com uma explicação curta.
Pedir poucos dados ao jogador, de acordo com a LGPD.
Manter a interface simples, com poucos cliques para iniciar a partida.

Fontes de pesquisa sugeridas

Relatórios e estudos: pesquisas sobre o mercado de games no Brasil (Pesquisa Game Brasil, Sioux/Go Gamers e ABRAGAMES) e estudos sobre gamificação na educação.
Concorrentes: os sites e aplicativos de Kahoot!, Quizizz e Perguntados, testados pela equipe.
Artigos acadêmicos: buscas no Google Scholar por "gamificação", "quiz educativo" e "engajamento em jogos educacionais".
Blogs e benchmarks internacionais: sites de game design e de UX de jogos, como Game Developer e Nielsen Norman Group.
Ferramentas: Google Scholar, Think with Google, Statista e a documentação oficial do C# e do .NET (Microsoft Learn) para a parte técnica.

2.3. Trilha do usuário

. Roteiro em 6 passos

1. Escolha um usuário e um objetivo. Lucas, aluno do ensino médio, quer jogar uma partida rápida de quiz no computador, avançar do nível fácil ao difícil e fazer uma boa pontuação, sem precisar fazer cadastro.

2. Marque o começo e o fim. A trilha começa antes do sistema, quando Lucas sente vontade de jogar algo rápido e divertido e abre o Tortuquiz. Termina depois do objetivo cumprido, quando ele vê o resultado final, consulta o histórico e decide jogar de novo para superar sua pontuação.

3. Liste os passos em ordem. Agrupei em 6 etapas:

Abrir o jogo e ler as instruções.

Informar o apelido ou entrar de forma anônima.

Escolher o modo de jogo e iniciar a partida.

Responder às perguntas, do nível fácil ao difícil.

Receber a pontuação e o feedback educativo.

Ver o resultado final e o histórico de partidas.

4. Para cada etapa, anote o que a pessoa faz, o que ela sente e onde ela trava.

Etapa 1. Lucas abre o aplicativo e lê o objetivo e as regras. Sente curiosidade. Trava se as instruções forem longas ou confusas.

Etapa 2. Lucas digita um apelido ou escolhe jogar anônimo. Sente receio de passar dados pessoais. Trava se o jogo exigir cadastro.

Etapa 3. Lucas escolhe o modo e aguarda o carregamento das perguntas do banco de dados. Sente ansiedade para começar. Trava se o carregamento demorar ou falhar.

Etapa 4. Lucas responde às perguntas e pode usar pistas. Sente desafio e empolgação. Trava nas perguntas difíceis e pode se frustrar.

Etapa 5. Lucas vê seus pontos, o progresso e uma explicação sobre a resposta. Sente motivação e aprendizado. Trava se não entender como a pontuação é calculada ou se o feedback for longo demais.

Etapa 6. Lucas vê o resumo da partida, as informações sobre as marcas/produtos e as partidas anteriores. Sente satisfação e vontade de melhorar. Trava se o resultado for pouco claro ou se o histórico não estiver disponível.

5. Transforme cada dor em oportunidade.

Instruções confusas: apresentar regras curtas e objetivas na tela inicial, com um botão "Iniciar" em destaque.

Exigência de cadastro: permitir apelido opcional ou início anônimo, sem pedir dados pessoais.

Demora no carregamento: mostrar um indicador de progresso e, em caso de erro, uma mensagem clara com a opção de tentar novamente.

Frustração nas perguntas difíceis: aumentar a dificuldade aos poucos e oferecer pistas.

Pontuação confusa ou feedback longo: deixar pontos e progresso sempre visíveis e dar explicações curtas após cada resposta.

Resultado ou histórico confuso: exibir um resumo visual com a Tortuguita e uma lista simples de partidas com data, pontuação e nível alcançado.

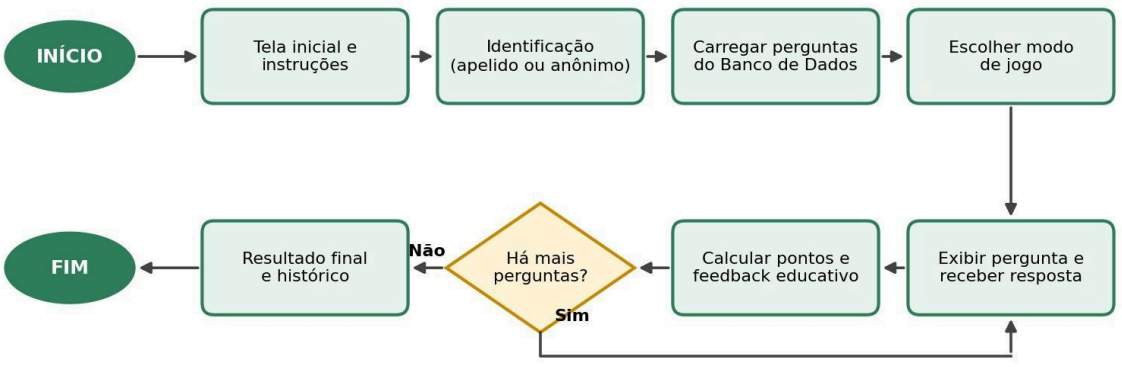
6. Valide com uma pessoa real. Vamos pedir a um colega que não conhece o projeto que use o Tortuquiz do início ao fim, sem ajuda, enquanto observamos em silêncio. Vamos anotar onde ele hesita, erra o caminho ou faz perguntas, pois o que ele faz vale mais do que o que imaginamos. Com base nisso, ajustamos telas, textos e regras antes da versão final.

Exemplo — Tortuquiz

Usuário: Lucas, ensino médio. Objetivo: jogar uma partida rápida de quiz, avançar do nível fácil ao difícil e fazer uma boa pontuação, sem cadastro.

Etapa	Descobrir	Identificar-se	Jogar	Aprender	Ver resultado
O que o jogador faz	Abre o app e lê as instruções	Digita um apelido ou joga anônimo	Escolhe o modo e responde às perguntas	Lê o feedback após cada resposta	Vê a pontuação e o histórico
Como ele se sente	Curioso	Receoso	Desafiado	Surpreso	Satisfeito
Onde ele trava	Instruções longas	Cadastro obrigatório	Perguntas difíceis demais	Feedback longo	Resultado confuso
Oportunidade	Regras curtas e botão "Iniciar" em destaque	Apelido opcional, sem dados pessoais	Dificuldade progressiva e pistas	Explicação curta e curiosidade sobre a marca	Resumo visual com a Tortuguita e histórico simples

2.4. Prototipagem



REQUISITOS DE SISTEMA

2.5. REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Exibir tela inicial e instruções
Descrição	O sistema apresenta a identidade do Tortuquiz, o objetivo do jogo, as instruções de uso e a opção de iniciar uma partida.
Entradas	Clique no botão "Iniciar".
Fonte	Jogador.
Saídas	Tela inicial com instruções; direcionamento para a tela de identificação.
Ação	Ao abrir o aplicativo, o sistema exibe a tela inicial com o nome do jogo, o objetivo e as regras. Ao clicar em "Iniciar", o sistema abre a tela de identificação do jogador.

RFS02	
Função	Identificar o jogador
Descrição	O sistema permite que o jogador informe um nickname/apelido ou inicie a partida de forma anônima, sem exigir dados pessoais.
Entradas	Nickname (texto) ou opção "Jogar anônimo".
Fonte	Jogador.
Saídas	Jogador identificado (apelido ou "Anônimo") para a partida.
Ação	O sistema exibe um campo para o apelido e a opção de jogar anônimo. Se o campo estiver vazio, o jogador é registrado como "Anônimo". Depois, o sistema segue para a carga do conteúdo.

RFS03	
Função	Carregar conteúdo do Banco de Dados
Descrição	O sistema recupera do Banco de Dados relacional as perguntas, alternativas, pistas, marcas/produtos e parâmetros de jogo.

RFS03	
Entradas	Solicitação de início de partida.
Fonte	Banco de Dados relacional.
Saídas	Lista de perguntas por nível (fácil, médio e difícil) com alternativas, pistas e informações das marcas/produtos.
Ação	Ao iniciar a partida, o sistema consulta o Banco de Dados e carrega o conteúdo em memória. Se houver falha na conexão, exibe uma mensagem de erro e permite tentar novamente.

RFS04	
Função	Selecionar o modo e executar o quiz
Descrição	O sistema permite escolher a mecânica principal (Quiz ou Descoberta por perguntas sucessivas) e apresenta as perguntas em ordem crescente de dificuldade.
Entradas	Modo de jogo escolhido; alternativa selecionada pelo jogador; uso de pista (opcional).
Fonte	Jogador e Banco de Dados.
Saídas	Pergunta atual com alternativas; confirmação da resposta escolhida.
Ação	Após a escolha do modo, o sistema exibe uma pergunta por vez, começando pelo nível fácil e avançando para médio e difícil. A cada resposta, o sistema registra a escolha e passa para a próxima pergunta.

RFS05		
Função	Calcular pontuação, exibir progresso e dar feedback educativo	
Descrição	O sistema calcula a pontuação conforme a regra documentada, mostra o progresso da partida e apresenta uma explicação educativa após cada resposta.	
Entradas	Resposta do jogador; nível da pergunta; regra de pontuação.	
Fonte	Jogador e Banco de Dados.	
Saídas	Pontuação atualizada; indicador de progresso (rodada, pergunta atual); indicação de acerto ou erro; explicação sobre o tema ou a marca/produto.	
Ação	O sistema compara a resposta com a alternativa correta, soma os pontos em caso de acerto (valor maior para perguntas mais difíceis), atualiza o progresso e exibe o feedback educativo antes da próxima pergunta.	

RFS05		

RFS06	
Função	Exibir resultado final e histórico de partidas
Descrição	Ao fim da partida, o sistema mostra o resumo do desempenho e armazena a partida para consulta posterior.
Entradas	Pontuação final, número de acertos e erros, identificação do jogador; solicitação de consulta ao histórico.
Fonte	Sistema (dados da partida) e Banco de Dados.
Saídas	Tela de resultado com pontuação, desempenho e informações sobre as marcas/produtos abordados; lista de partidas anteriores (data, jogador, pontuação).
Ação	Quando acabam as perguntas, o sistema exibe o resumo e grava a partida no Banco de Dados. O jogador pode consultar o histórico ou iniciar uma nova partida.

2.6. REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RFS01	
Função	Usabilidade da interface
Descrição	A interface do Tortuquiz deve ser simples e intuitiva, para que qualquer jogador consiga iniciar e jogar sem ajuda.
Entradas	Interações do jogador com as telas (cliques e seleção de alternativas).
Fonte	Jogador.
Saídas	Telas claras, com botões visíveis e mensagens objetivas
Ação	O sistema deve permitir iniciar uma partida em até 3 cliques a partir da tela inicial, usar textos curtos e manter o mesmo padrão visual (cores, botões e a mascote Tortuguita) em todas as telas.

RFS02	
Função	Desempenho do sistema

Descrição	O sistema deve responder rapidamente às ações do jogador e carregar o conteúdo sem demora perceptível.
Entradas	Início da partida e resposta a cada pergunta.
Fonte	Jogador e Banco de Dados.
Saídas	Perguntas, pontuação e feedback exibidos sem atrasos.
Ação	O sistema deve carregar o conteúdo do Banco de Dados em até 5 segundos e exibir cada nova pergunta e cada feedback em até 2 segundos após a ação do jogador.

RFS03	
Função	Compatibilidade e portabilidade
Descrição	O aplicativo deve ser desenvolvido em C# como aplicação Desktop e funcionar em computadores com Windows.
Entradas	Execução do aplicativo no computador do jogador.
Fonte	Sistema operacional e ambiente .NET
Saídas	Aplicativo em funcionamento, sem erros de execução.
Ação	O sistema deve executar em Windows 10 ou superior, com o .NET instalado, e funcionar nas resoluções de tela mais comuns de computadores desktop e notebooks.

RFS04	
Função	Segurança e privacidade dos dados
Descrição	O sistema não deve exigir nem armazenar dados pessoais desnecessários, e o acesso ao Banco de Dados deve ser protegido.
Entradas	Apelido do jogador e consultas ao Banco de Dados.
Fonte	Jogador e aplicativo.
Saídas	Dados armazenados apenas com o apelido (ou "Anônimo") e conexão protegida.
Ação	O sistema deve guardar somente o apelido e os dados da partida, acessar o Banco de Dados com usuário e senha e usar consultas parametrizadas para evitar SQL Injection.

RFS05

Função	Confiabilidade e tratamento de erros
Descrição	O sistema deve tratar falhas sem travar e preservar os dados das partidas já realizadas.
Entradas	Falhas de conexão com o Banco de Dados ou entradas inválidas do jogador.
Fonte	Banco de Dados e jogador.
Saídas	Mensagens de erro claras e histórico preservado.
Ação	Em caso de erro, o sistema deve exibir uma mensagem compreensível, permitir tentar novamente sem fechar o aplicativo e não perder o histórico de partidas já salvas.

RFS06	
Função	Manutenibilidade do sistema
Descrição	O código e o Banco de Dados devem ser organizados para facilitar correções e a inclusão de novo conteúdo.
Entradas	Novas perguntas, pistas e marcas/produtos cadastrados no Banco de Dados; alterações no código.
Fonte	Equipe de desenvolvimento.
Saídas	Sistema atualizado sem necessidade de reescrever o código.
Ação	O sistema deve permitir incluir novo conteúdo diretamente no Banco de Dados, sem alterar o código, e o código deve separar interface, regras de negócio e acesso a dados, com comentários nas partes principais.

DIAGRAMAS DE UML

Casos de uso:

Caso de Uso 1: Identificar Jogador

Código/Identificação: UC01

Ator Principal: Jogador

Descrição: O sistema coleta as informações iniciais do usuário para iniciar a sessão do quiz.

Fluxo Principal:

O sistema exibe a tela de abertura com a arte e as instruções do jogo.

O sistema solicita que o jogador digite seu *e-mail e senha*.

O sistema apresenta um menu interativo com as faixas etárias disponíveis

O jogador seleciona sua faixa etária usando o comando e confirma com Enter.

O sistema armazena os dados e avança para o loop principal do quiz.

Caso de Uso 2: Responder Pergunta do Quiz

Código/Identificação: UC02

Ator Principal: Jogador

Descrição: O jogador interage com as perguntas de verdadeiro ou falso..

Fluxo Principal:

O sistema exibe o enunciado da questão atual.

O sistema apresenta as opções de escolha (*Verdadeiro*, *Falso*) e a opção de saída (*TERMINAR*).

O jogador seleciona uma das opções.

O sistema valida se a resposta escolhida corresponde ao gabarito oficial.

Caso de Uso 3: Visualizar Resultado Final e Classificação

Código/Identificação: UC03

Ator Principal: Jogador

Descrição: O sistema calcula a pontuação total do usuário ao término das questões e exibe o seu desempenho consolidado.

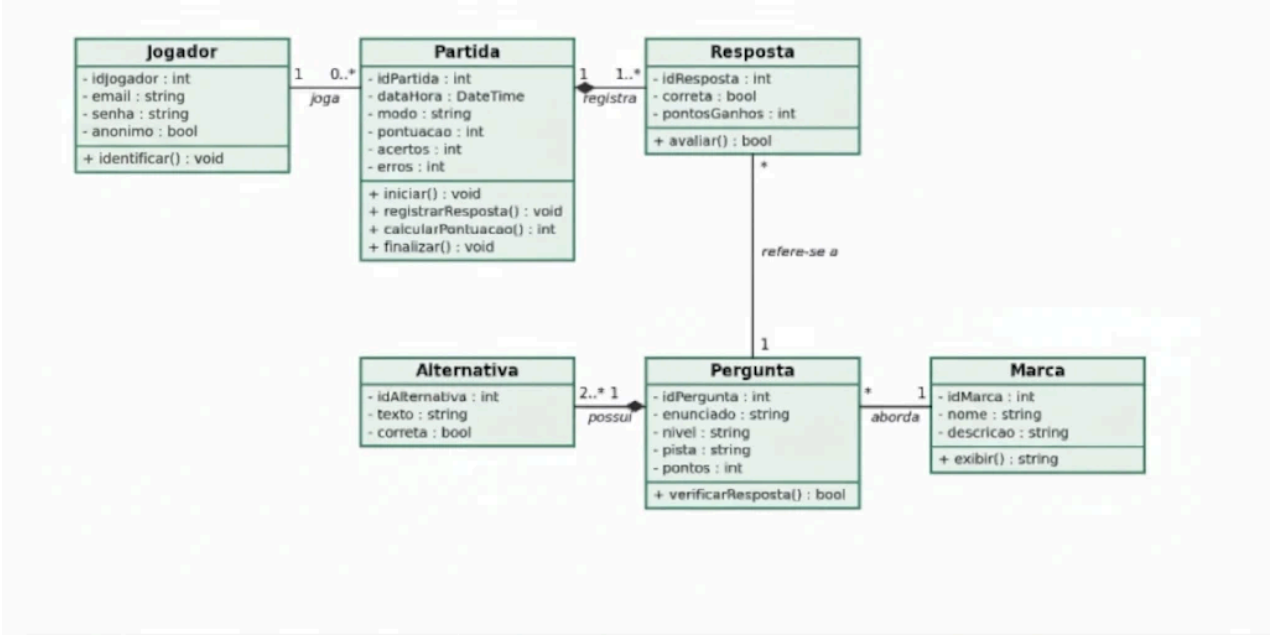
Fluxo Principal:

O sistema verifica que todas as questões foram respondidas.

O sistema soma a quantidade de acertos obtidos em cada nível de dificuldade (Fácil, Médio e Difícil).

O sistema atribui uma classificação de desempenho ao jogador com base nos pontos acumulados (ex: *Tortuguita Mestre*, *Tortuguita Sábia*, *Tortuguita Aprendiz* ou *Tortuguita Noob*).

O sistema exibe na tela o resumo final contendo o nome do jogador, o acerto detalhado por nível, a pontuação total e o título de classificação alcançado.



Especificação estrutura de uma rotina do sistema

Sistema/Software: Tortuquiz (aplicativo Desktop em C#)

Função: Calcular pontuação e atualizar o progresso da partida.

Descrição: A rotina compara a alternativa escolhida pelo jogador com a alternativa correta da pergunta. Em caso de acerto, soma à pontuação os pontos do nível da pergunta (fácil = 10, médio = 20, difícil = 30). Depois atualiza o número de acertos, de erros e o progresso da partida.

Entradas: Pergunta atual (com nível), alternativa escolhida pelo jogador e estado atual da partida (pontuação, acertos, erros e perguntas respondidas).

Fonte: Jogador (alternativa escolhida) e Banco de Dados (pergunta e alternativa correta).

Saídas: Indicação de acerto ou erro, pontos ganhos, pontuação total atualizada e progresso (pergunta atual de um total).

Destino: Tela do quiz (exibição da pontuação, do progresso e do feedback) e objeto Partida (usado depois para o resultado final e o histórico).

Ação:

- Receber a pergunta e a alternativa escolhida.
- Verificar se a alternativa escolhida é a correta.
- Se acertar, obter os pontos conforme o nível da pergunta e somar a pontuação.
- Se errou, manter a pontuação e registrar o erro.
- Atualizar acertos, erros e o contador de perguntas respondidas.
- Devolver o resultado para a tela exibir o feedback educativo.

Requisitos: RFS04 (Selecionar o modo e executar o quiz) e RFS05 (Calcular pontuação, exibir progresso e dar feedback educativo).

Pré-condição: A partida foi iniciada, as perguntas foram carregadas do Banco de Dados e há uma pergunta exibida ao jogador, com uma alternativa selecionada.

Pós-condições: A pontuação, os acertos, os erros e o progresso da partida estão atualizados, e o resultado da resposta está disponível para exibição.

Efeitos colaterais: A tela do quiz é atualizada (pontuação e barra de progresso). Se a pergunta era a última, a partida passa para a etapa final.

5.2 – Apresentar a codificação da rotina

```
public class Jogador
{
    public int IdJogador { get; private set; }
    public string Email { get; private set; }
    public string Senha { get; private set; }

    public Jogador(string email, string senha)
    {
        Email = email;
        Senha = senha;
    }

    public bool Identificar(string emailDigitado, string senhaDigitada)
    {
        return Email == emailDigitado && Senha == senhaDigitada;
    }
}

public class Partida
{
    public int Pontuacao { get; private set; }
    public int Acertos { get; private set; }
    public int Erros { get; private set; }
    public int PerguntasRespondidas { get; private set; }
    public int TotalPerguntas { get; private set; }
    public Jogador JogadorAtual { get; private set; }

    public Partida(Jogador jogador, int totalPerguntas)
    {
        JogadorAtual = jogador ?? throw new ArgumentNullException("Jogador é obrigatório.");
        TotalPerguntas = totalPerguntas;
    }

    // Retorna os pontos ganhos na resposta (0 se errou)
    public int RegistrarResposta(Pergunta pergunta, Alternativa escolhida)
    {
        if (pergunta == null || escolhida == null)
            throw new ArgumentException("Pergunta e alternativa são obrigatórias.");
```

```
        int pontosGanhos = 0;

        if (escolhida.Correta)
        {
            pontosGanhos = ObterPontosPorNivel(pergunta.Nivel);
            Pontuacao += pontosGanhos;
            Acertos++;
        }
        else
        {
            Erros++;
        }

        PerguntasRespondidas++;
        return pontosGanhos;
    }

    private int ObterPontosPorNivel(string nivel)
    {
        switch (nivel.ToLower())
        {
            case "facil": return 10;
            case "medio": return 20;
            case "dificil": return 30;
            default:
                throw new ArgumentException("Nível de pergunta inválido.");
        }
    }

    public bool TemMaisPerguntas()
    {
        return PerguntasRespondidas < TotalPerguntas;
    }
}
```

ARQUITETURA DO SISTEMA

6.1. Visão Geral do Sistema

- **Nome do Sistema**: Tortuquiz
- **Objetivo do Sistema**: Oferecer um quiz de entretenimento com a temática "Tortuguita", no qual o jogador avança por perguntas de nível fácil, médio e difícil, acumula pontos,
  recebe feedback educativo e conhece marcas e produtos ao longo da partida.
- **Público-Alvo**: Jovens e adultos que gostam de jogos de perguntas e respostas, especialmente estudantes do ensino médio e universitários.
- **Principais Funcionalidades**: Tela inicial com instruções; identificação por apelido ou anônima; carga de perguntas do Banco de Dados; seleção do modo de jogo; execução do quiz por níveis; cálculo de pontuação e progresso; feedback educativo; resultado final; histórico de partidas.

6.2. Arquitetura Geral

O sistema adota a **Arquitetura em Camadas (Layered Architecture)**, com separação clara entre apresentação, lógica de negócio e acesso a dados. Cada camada se comunica apenas com a camada imediatamente abaixo. Isso facilita a manutenção e permite incluir novas perguntas ou alterar a interface sem mexer nas regras do jogo.

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: C#, Windows Forms (.NET).
- **Responsabilidades**:
  - Interface com o jogador (telas inicial, identificação, quiz, resultado e histórico).
  - Validação inicial dos dados digitados (por exemplo, tamanho do apelido).
  - Comunicação com a camada de negócio por chamadas de métodos, sem acesso direto ao banco.

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: C# (.NET).
- **Responsabilidades**:
  - Coordenar o fluxo da partida (iniciar, responder, finalizar).
  - Receber as ações da interface e chamar as regras de negócio.
  - Comunicação com a camada de dados.
  - Tratamento de erros e mensagens para o jogador.

6.3.3. Camada de Domínio (Negócio)

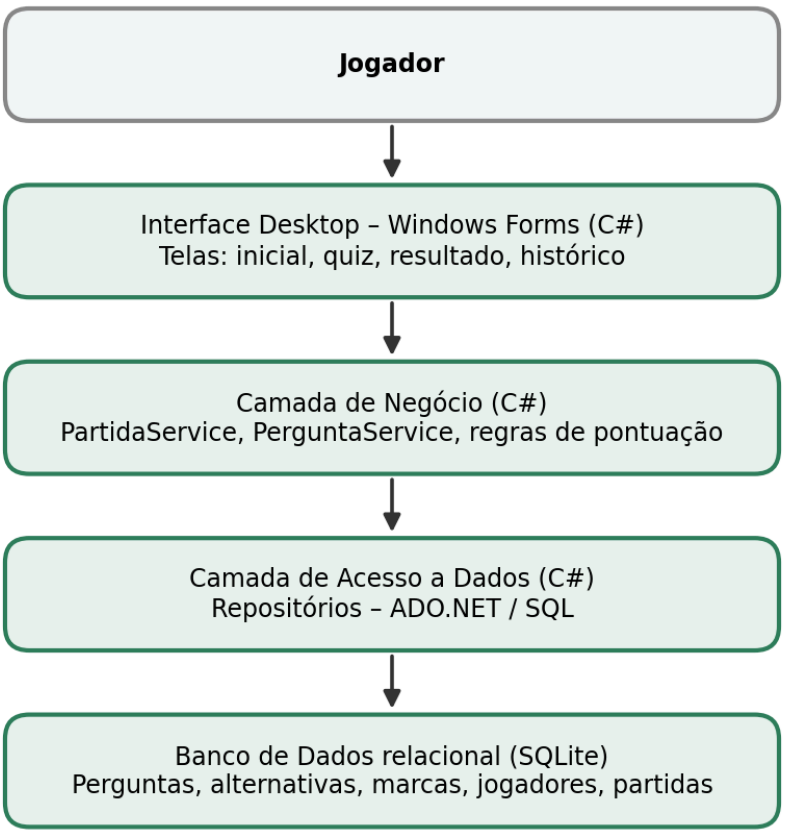
- **Descrição**:
  - Contém as regras do jogo e os modelos do domínio.
  - Independente da interface e do banco de dados.

- **Componentes**:
  - Entidades: Jogador, Partida, Pergunta, Alternativa, Resposta e Marca.
  - Casos de uso: iniciar partida, responder pergunta, consultar histórico.
  - Regras de validação: cálculo de pontuação por nível, verificação da resposta correta e controle de progresso.

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: SQLite, com acesso via ADO.NET
- **Responsabilidades**:
  - Armazenamento e recuperação de perguntas, alternativas, marcas, jogadores e partidas.
  - Scripts de criação do banco e carga inicial de perguntas.

6.4. Diagrama da Arquitetura



6.5. Organização de Pastas e Módulos

/Tortuquiz

/Apresentacao (formulários Windows Forms) FrmInicial.cs

FrmIdentificacao.cs

FrmQuiz.cs

FrmResultado.cs

FrmHistorico.cs

/Negocio (regras e serviços) PartidaService.cs

PerguntaService.cs

/Dominio

(entidades) Jogador.cs

Partida.cs

Pergunta.cs

Alternativa.cs

Resposta.cs

Marca.cs

/Dados (acesso ao

banco) Conexao.cs

PerguntaRepository.cs

PartidaRepository.cs

/Banco

schema.sql (criação das tabelas)

carga_inicial.sql (perguntas e marcas)

/Recursos (imagens da Tortuguita,

sons) Program.cs

6.6. Ambiente e Implantação

- **Ambientes**: desenvolvimento (computadores da equipe, com Visual Studio, .NET e SQLite) e homologação (testes do aplicativo compilado em outro computador, simulando o ambiente do jogador). Como o Tortuquiz é um aplicativo Desktop local, a implantação em produção é a distribuição do executável instalado no computador do jogador, e não há um servidor de produção.
- **Contêineres**: não utilizados. Como o aplicativo roda localmente e usa um banco SQLite em arquivo, não há necessidade de Docker.
- **CI/CD**: GitHub Actions, para compilar o projeto e executar os testes a cada alteração enviada ao repositório. A entrega é manual: a equipe gera o executável (publicação do
.NET) e disponibiliza o arquivo ou o instalador.
- **Hospedagem**: não há hospedagem em nuvem. O repositório fica no GitHub, e o executável é distribuído por arquivo compartilhado (por exemplo, GitHub Releases ou Google Drive).

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.